

CE 488/5588: Applied AI for Engineering

Topic 15: Introduction to Large Language Models

This chapter begins a new part of the course. Earlier topics focused on machine learning and deep learning as predictive or generative tools. We now turn to language-centered foundation models. The motivation is not only that modern large language models (LLMs) are impressive, but that engineering work is already organized around language-like artifacts: design codes, specifications, reports, spreadsheets, calculation sheets, tables, plots, CAD notes, emails, and source code. If these materials can be searched, summarized, connected, and reasoned over faster, then analysis and design may be accelerated even when they are not fully automated.

1 Why This Topic Matters for Engineers

Much of modern engineering is document-heavy. Before a design is finalized, an engineer may need to read standards, compare alternatives, inspect previous reports, interpret tables, examine calculations, and coordinate across software tools. In practice, this means that engineering knowledge is often stored in semi-structured text long before it becomes a final number or drawing.

LLMs matter because they operate directly on this kind of material. They do not replace mechanics, physics, or engineering judgment. Instead, they provide a new interface for interacting with technical knowledge. A well-designed workflow can help an engineer:

- search large bodies of technical documents in natural language;
- summarize long reports or standards before deeper review;
- compare alternative design assumptions across multiple sources;
- draft code, scripts, and analysis notebooks; and
- organize planning or research tasks that would otherwise require many manual steps.

The practical consequence is important. The first large impact of LLMs on engineering is likely not full automation. It is accelerated interaction with information. For knowledge-intensive work, that alone may be transformative.

Engineering perspective: The near-term power of LLMs is that they compress the time needed to move from a question to the relevant pieces of text, code, and prior knowledge. That can accelerate engineering analysis even when the final design decision remains human.

Learning Objectives

After completing this chapter, you should be able to:

- explain why language models have become relevant to engineering workflows;
- summarize the main stages in the history of natural language processing (NLP), from symbolic methods to neural models;
- explain why modern LLMs succeeded where earlier NLP systems struggled;
- describe the role of tokenization, embeddings, and high-dimensional vector representations;
- explain how a sentence or paragraph can be retrieved from a vector store; and

- interpret a basic retrieval pipeline in terms of chunking, embeddings, similarity, and metadata lookup.

2 A Motivating View: Language as Engineering Infrastructure

When people hear “language model,” they often think first of a chatbot. That view is too narrow for engineering. Language is the surface layer of many engineering systems:

- building codes and specifications are written as formal documents;
- inspection logs and maintenance histories are textual records;
- spreadsheets and tables encode structured decisions and assumptions;
- source code is executable language; and
- research papers, standards, and design memos are how engineering knowledge is transmitted.

An LLM can therefore be seen as a model for interacting with the information layer of engineering work. That does not mean the model understands the world in the same way a human expert does. It means the model can map between different textual expressions, retrieve related concepts, and generate candidate responses in ways that are often useful.

This broader view also helps explain why discussions of advanced AI often invoke the idea of artificial general intelligence (AGI). Human knowledge is heavily mediated by language. If a system becomes increasingly capable of operating over that language, it begins to approximate some aspects of general-purpose intellectual work. Whether that should count as AGI remains debatable, but it is enough to motivate careful study in engineering education.

Example: a bridge assessment workflow. Suppose an engineer is reviewing a bridge rehabilitation project. The relevant information may include inspection notes, prior load-rating reports, AASHTO code clauses, spreadsheets of section properties, emails about field changes, and Python scripts for analysis. None of these alone is “the design,” yet together they form the knowledge environment in which the design is produced. LLM-based systems become useful precisely because they can help navigate this environment.

3 Why NLP Was Historically Difficult

Natural language processing has always been hard because language is ambiguous, context-dependent, and combinatorial. The same word can have different meanings in different domains, and the same idea can be expressed in many different ways. For engineers, an additional difficulty is that technical language mixes prose with equations, tables, abbreviations, units, and references to external standards.

Researchers therefore spent decades trying to make language computationally manageable. The history of NLP can be understood as a sequence of partial successes.

3.1 Rule-Based Beginnings

In the early years, NLP was largely symbolic. Researchers wrote grammars, dictionaries, and explicit rules for translation or parsing. Warren Weaver’s 1949 memorandum is often cited as an early milestone because it framed machine translation as an information-processing problem rather than a purely linguistic one.

One influential probabilistic framing was the noisy-channel view of translation:

$$\hat{e} = \arg \max_e P(e \mid f) = \arg \max_e P(f \mid e)P(e), \quad (1)$$

where f is a sentence in a foreign language and e is a candidate sentence in English. Even though early systems were not yet modern machine learning models, this equation already hinted at two ideas that would remain central: a model of fluent language $P(e)$ and a model linking one expression to another $P(f | e)$.

Rule-based systems were conceptually attractive because they were interpretable. If the system failed, one could inspect the rules. However, this approach had major weaknesses:

- building the rules required enormous manual effort;
- exceptions and edge cases accumulated quickly; and
- systems were brittle outside carefully designed domains.

3.2 Statistical NLP

As larger text corpora became available in the 1980s and 1990s, researchers shifted toward statistical models. Instead of hand-coding all linguistic structure, they estimated probabilities from data. A classic example is the n -gram language model,

$$P(w_1, \dots, w_T) \approx \prod_{t=1}^T P(w_t | w_{t-n+1}, \dots, w_{t-1}), \quad (2)$$

which approximates the probability of a sequence by looking only at a short local window.

These methods represented major progress. They allowed systems to learn from observed language patterns rather than only from expert-written rules. Yet they still had important limits:

- context windows were short;
- meaning was represented only indirectly through local statistics; and
- performance degraded when long-range dependencies mattered.

3.3 Neural Sequence Models Before Transformers

RNNs, LSTMs, and sequence-to-sequence models brought another leap. These neural models learned continuous representations and handled variable-length sequences more naturally than earlier statistical systems. If a hidden state is denoted by $\mathbf{h}_t$, the recurrent update has the general form

$$\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1}), \quad (3)$$

so the model carries information from one step to the next.

Still, pre-transformer neural NLP faced two persistent problems. First, long-context modeling was difficult. Second, large-scale training remained inefficient because recurrence limited parallel computation. Attention mechanisms improved matters, but the field still lacked a general architecture that scaled cleanly.

Table 1: A compact timeline of major NLP eras.

Era	Representative methods	Main limitation
1950s–1970s	Rule-based translation, symbolic parsing, formal grammars	Too much expert labor; brittle outside narrow domains
1980s–2000s	N-grams, HMMs, statistical machine translation	Short context and weak semantic representation
1990s–2010s	RNNs, LSTMs, seq2seq with attention	Long-range dependence and scaling remained difficult
2017–today	Transformers, GPT-style pretraining, foundation models	Compute cost, reliability, alignment, and interpretability

Key lesson from history: NLP was not blocked by a lack of interest. It was blocked by representation and scale. Language requires both a useful representation of meaning and enough data and compute to learn that representation well.

4 Why Modern LLMs Succeeded

Modern LLMs did not emerge from a single breakthrough. Their success came from several ideas reinforcing one another.

4.1 The Transformer

The 2017 Transformer architecture replaced recurrence with self-attention. This change mattered because it allowed models to compare each token with other tokens in the sequence more directly and to train efficiently on parallel hardware.

At a high level, attention answers the question: when interpreting the current token, which other tokens in the context matter most? For an input matrix $\mathbf{X} \in \mathbb{R}^{T \times d}$, we compute queries, keys, and values:

$$\mathbf{Q} = \mathbf{XW}^Q, \quad \mathbf{K} = \mathbf{XW}^K, \quad \mathbf{V} = \mathbf{XW}^V, \quad (4)$$

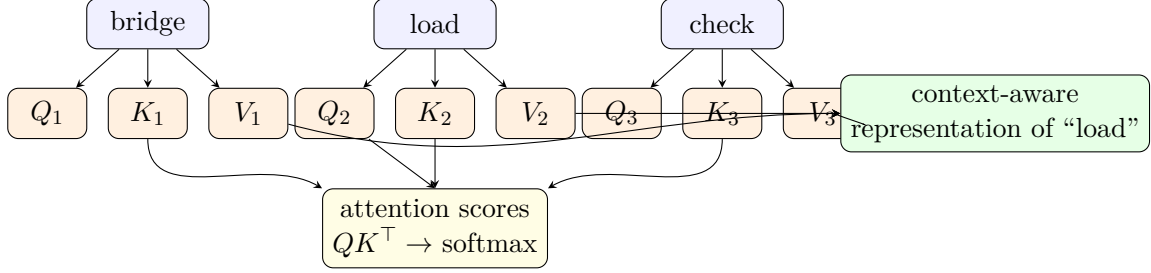
and then the attention output is

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}}\right) \mathbf{V}. \quad (5)$$

This equation is worth pausing on. The dot product $\mathbf{QK}^\top$ measures which tokens are relevant to which others. The softmax turns those relevance scores into weights. The weighted sum of values then updates the representation of each token using information from the whole context.

4.2 GPT-Style Generative Pretraining

GPT-style models use a simple but powerful training objective: predict the next token from the tokens that came before it. If the token sequence is $x_1, \dots, x_T$, the objective is to



The token “load” forms a query, compares itself against keys from all tokens, and uses the resulting weights to mix value vectors into a contextualized representation.

Figure 1: Schematic of self-attention. Each token forms query, key, and value vectors; attention weights then determine how strongly information from other tokens contributes to its updated representation.

maximize

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_{<t}), \quad (6)$$

or equivalently to minimize the negative log-likelihood

$$\mathcal{L}(\theta) = - \sum_{t=1}^T \log P_{\theta}(x_t \mid x_{<t}). \quad (7)$$

This objective appears modest, yet when applied at large scale it forces the model to learn syntax, semantics, style, factual associations, and many latent task patterns. This is the central idea behind **generative pretraining**: train once on massive corpora, then adapt the same model to many tasks through prompting, instruction tuning, or downstream tooling.

5 From Text to Tokens

To understand why LLMs are useful, we need one core conceptual shift: computers do not work directly with words as meanings. They work with numerical representations.

5.1 Tokenization

Before text can be processed, it is split into smaller units called **tokens**. A token may be a whole word, part of a word, punctuation, or even a short character sequence. Modern tokenizers are usually data-driven and balance vocabulary size against flexibility.

For example, the sentence

`"Design loads must be checked carefully."`

might be broken into tokens such as

`[Design, loads, must, be, checked, carefully, .].`

For rarer technical words, the tokenizer may split the word into smaller pieces. This is useful because engineering language contains many compound terms, abbreviations, and domain-specific symbols. A word such as

`"geotechnical"`

might be represented by pieces such as

`[geo, technical]` or `[ge, otech, nical],`

depending on the tokenizer.

After tokenization, each token receives an integer ID. If token x_i is represented by a one-hot vector, then the transition to an embedding layer can be written as

$$\mathbf{e}_i = E \text{ one_hot}(x_i), \tag{8}$$

where E is the embedding matrix.

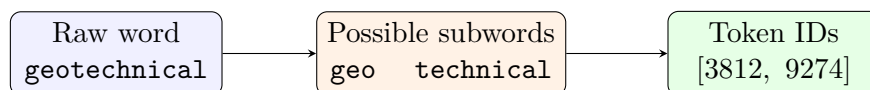
5.2 A Concrete Byte-Pair Encoding Intuition

A popular family of tokenizers is based on subword merges such as byte-pair encoding (BPE). The basic idea is simple: start from smaller units, count frequent adjacent pairs, and merge the most useful ones. A stylized example is shown in Table 2.

Table 2: A simplified intuition for subword merging.

Step	Frequent pair	Possible merge result
1	d + esign	design
2	load + s	loads
3	deflect + ion	deflection
4	reinforce + ment	reinforcement

The exact algorithm is more detailed than this table suggests, but the educational point is clear: the tokenizer builds a vocabulary of useful pieces. Frequent whole words may become one token; rarer or specialized terms may remain split into multiple subwords.



Subword tokenization keeps common engineering terms compact when possible, while still allowing rare technical vocabulary to be represented from reusable pieces.

Figure 2: Subword tokenization of a technical term. Modern tokenizers often break specialized vocabulary into reusable pieces before mapping those pieces to integer token IDs.

Why tokenization matters in engineering. A phrase like “ACI 318 load combination” contains ordinary English, a code name, and a domain-specific term. Tokenization determines how efficiently the model represents that phrase. This affects cost, context-window usage, and sometimes even output quality.

6 From Tokens to Vectors

6.1 Embeddings

Each token is mapped to a vector of real numbers, called an **embedding**. The embedding is not manually assigned. It is learned during training so that tokens appearing in similar contexts tend to acquire related vector representations.

Mathematically, if the vocabulary has size $|\mathcal{V}|$ and the embedding dimension is d , then the embedding table is a matrix

$$E \in \mathbb{R}^{|\mathcal{V}| \times d}. \quad (9)$$

Looking up a token returns one row of this matrix.

Token versus embedding. A token is a discrete symbol. An embedding is its continuous numerical representation. Tokenization discretizes language; embeddings place that language in a vector space where similarity can be computed.

6.2 Why High-Dimensional Spaces Matter

Embeddings live in high-dimensional spaces, often with hundreds or thousands of coordinates. This sounds abstract, but the idea is simple. A high-dimensional vector can encode many weak signals at once: syntax, topic, tone, domain, and contextual associations.

In such spaces, tokens or passages with related meanings tend to lie near one another under a similarity measure such as cosine similarity:

$$\text{cos_sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}. \quad (10)$$

This is why embedding spaces can support semantic search. A query about “steel design loads” may retrieve passages using related but not identical wording such as “load combinations for structural steel members.”

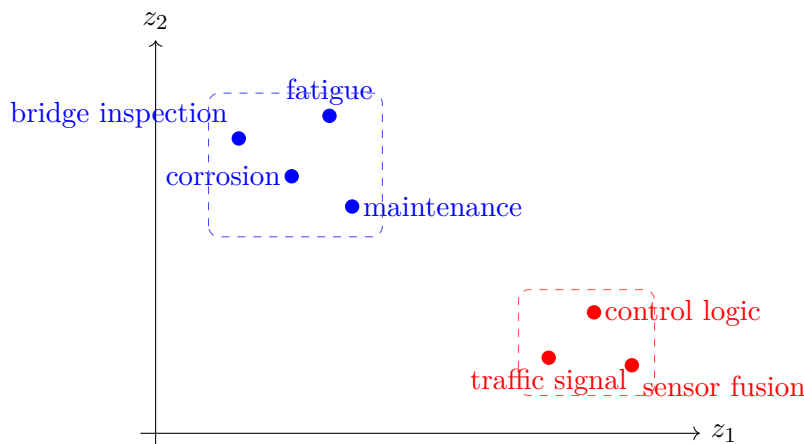


Figure 3: Low-dimensional view of an embedding space. In practice, embeddings live in hundreds or thousands of dimensions, but the same intuition applies: semantically related items tend to cluster near one another.

Geometric intuition. Imagine a library in which documents are arranged not alphabetically but by meaning. Documents about bridge inspection, fatigue, corrosion, and maintenance would cluster near one another even if they use different vocabulary. An embedding space tries to create that kind of geometry numerically.

6.3 A Minimal Workflow View

The front end of an LLM pipeline can be summarized as follows:

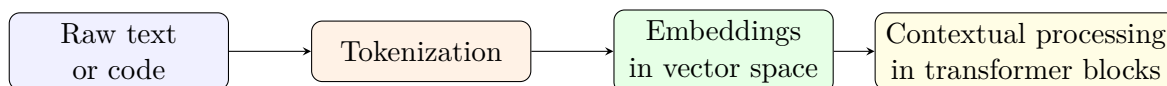


Figure 4: Front-end representation pipeline for an LLM. Topic 15 focuses on the transition from raw language to token IDs to embeddings, because that representation layer makes later contextual reasoning and retrieval possible.

In this chapter, the main emphasis is on the first half of that pipeline: how text becomes vectors and why those vectors are useful.

7 Sentence and Document Embeddings

So far we have discussed token embeddings. In many engineering workflows, however, we do not want to compare single tokens. We want to compare larger chunks: sentences, paragraphs, table descriptions, code blocks, or full report sections.

The high-level strategy is to compute a representation for a larger unit of text. Conceptually, if a chunk contains token embeddings $\mathbf{e}_1, \dots, \mathbf{e}_m$, one simple aggregation idea is

$$\mathbf{s} = \frac{1}{m} \sum_{i=1}^m \mathbf{e}_i, \quad (11)$$

though modern embedding models use more sophisticated contextual pooling than a simple mean.

The important teaching point is not the exact architecture. It is that an entire sentence or paragraph can also be mapped to a vector, allowing semantic comparison between chunks of text rather than only between words.

Table 3: Example chunking choices for engineering documents.

Source material	Possible chunk
Design code PDF	one clause or one paragraph
Structural report	one subsection or executive-summary paragraph
Spreadsheet note export	one row with neighboring notes
Python script	one function or one docstring-plus-function block
Inspection log	one observation entry

Chunking matters because it determines what the retrieval system can return. If chunks are too small, meaning is fragmented. If they are too large, retrieval becomes noisy and expensive.

8 Vector Stores and Retrieval

Once documents, paragraphs, code blocks, or table descriptions are converted into embeddings, those vectors can be stored and searched. A **vector store** is a database or indexing system designed for this purpose.

The basic pipeline is:

1. break a document collection into chunks;
2. embed each chunk as a vector;
3. store the vector together with metadata such as source file, page, section, and original text; and
4. given a new query, embed the query and retrieve nearby vectors.

This is useful because exact keyword matching is often too weak for technical work. An engineer may ask about “serviceability limits for deflection” while the relevant document uses a different phrase. Vector retrieval can still surface the right material because it compares meanings rather than exact strings.

8.1 Similarity Search

Suppose a query embedding is $\mathbf{q}$ and stored chunk embeddings are $\mathbf{v}_1, \dots, \mathbf{v}_N$. A basic retrieval score is

$$s_i = \text{cos_sim}(\mathbf{q}, \mathbf{v}_i), \quad i = 1, \dots, N, \quad (12)$$

and the system returns the top- k chunks:

$$\mathcal{R}_k(\mathbf{q}) = \text{TopK}_{i=1, \dots, N} s_i. \quad (13)$$

Operational systems may use dot product, Euclidean distance, approximate nearest-neighbor indexing, reranking, or hybrid keyword-plus-vector search. But the educational

core remains the same: represent the query and documents in the same vector space, then retrieve neighbors.

8.2 Keyword Search versus Semantic Search

It is useful to distinguish semantic retrieval from ordinary keyword retrieval. Keyword search asks whether the query words appear in the document. Semantic search asks whether the meanings are close in the embedding space. In practice, engineering systems often need both.

Table 4: Keyword search and semantic search play different roles.

Query type	Keyword search is strong when	Semantic search is strong when
Exact clause lookup	the exact code citation or phrase is known	wording varies across documents
Prior project search	repeated terminology is consistent	similar ideas are expressed with different phrases
Spreadsheet or note lookup	identifiers and labels are exact	assumptions are paraphrased in prose
Research support	proper nouns and exact terms matter	concept-level similarity matters more than shared words

For this reason, many production systems use **hybrid retrieval**: exact matching for precision and vector similarity for semantic recall. That combination is especially natural in engineering, where code names, member labels, project IDs, and equations often require exact lookup, while design reasoning and narrative explanations benefit from semantic search.

8.3 How Can a Sentence Be Retrieved from a Vector Store?

This is one of the most important conceptual questions in this topic. A vector store does not literally store meaning in words. It stores numerical vectors *and* references back to the original text.

In practice, each stored item looks conceptually like this:

ID	Vector	Source metadata	Original chunk text
17	$\mathbf{v}_{17}$	Report A, p. 42	“Deflection under service load shall...”
18	$\mathbf{v}_{18}$	Report B, p. 11	“Load combinations for steel beams...”
19	$\mathbf{v}_{19}$	Code note	“Drainage layer prevents hydrostatic...”

When the query vector is close to $\mathbf{v}_{17}$, the system does not return the vector itself to the human user. It uses the stored ID and metadata to return the associated chunk text. In other words, the vector is the search key, while the original sentence or paragraph remains the human-readable payload.

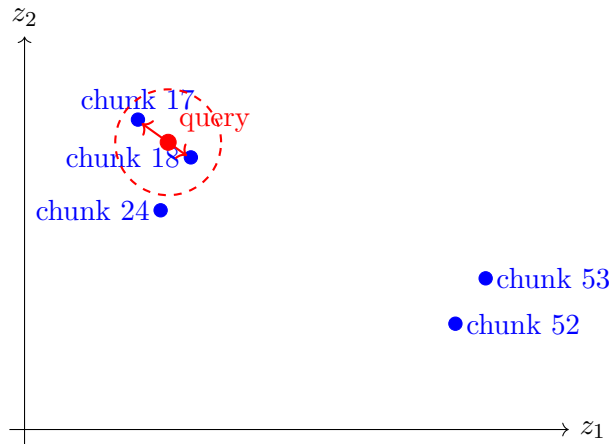


Figure 5: Nearest-neighbor retrieval in an embedding space. The query is embedded into the same vector space as the stored chunks, and the nearest chunk IDs are then used to recover the original sentences or paragraphs.

8.4 A Small Numerical Example

Consider a simplified query and three stored chunks. Suppose the query is

$$q = \text{"serviceability limit for beam deflection"},$$

and the cosine similarities are computed as shown in Table 5.

The system returns the first chunk because its vector lies nearest to the query vector

Table 5: A toy retrieval example.

Stored chunk	Cosine similarity	Interpretation
“Deflection under service loads shall remain within specified limits.”	0.92	Highly relevant
“Ultimate strength combinations control flexural resistance.”	0.67	Related but not the best match
“Retaining wall drainage layers reduce pore pressure.”	0.18	Largely unrelated

under the chosen metric. The essential point is that semantic retrieval works even if the exact phrase “beam deflection” does not appear in the returned sentence.

Important caution: A vector store does not make an LLM correct. It only improves the chance that the model sees relevant material. The final answer can still be incomplete, misleading, or wrong if the retrieved context is poor or if the model interprets it badly.

8.5 Why Metadata Matters

Metadata is the bridge back to the engineering source of truth. A vector alone is not enough. Engineers usually need to know:

- which document the retrieved chunk came from,
- what page or section it belongs to,
- whether it came from a design code, internal report, spreadsheet note, or source code file, and
- whether the source is current, approved, or superseded.

Without metadata, a system may retrieve semantically similar text but fail to support traceability, verification, or auditability.

Engineering example. Suppose a firm has thousands of PDF reports, specifications, code excerpts, and spreadsheet notes. A vector store can index chunks from those sources so that a query such as “find examples of retaining wall drainage design assumptions in prior projects” retrieves semantically related passages even when the exact wording varies across projects. The useful answer is not only the retrieved text; it is also the project name, report section, and page where that text came from.

9 Scale and Modern LLM Training

9.1 Scale

Large datasets, specialized hardware, and engineering improvements in distributed training made it possible to train models with billions of parameters. Empirically, larger models trained on more data often show smoother improvements in language tasks, coding tasks, and question answering. This observation is one reason modern AI discussions frequently emphasize scaling laws.

9.2 What Does “7B” or “70B” Mean?

In public discussions, the size of an LLM is often summarized by the number of **parameters**. A model described as “7B” has roughly seven billion trainable weights; “70B” means roughly seventy billion; and “1T” means roughly one trillion. This shorthand is imperfect, but it remains the most common public indicator of model scale.

Students should interpret this number carefully. Parameter count is not the same as intelligence, and it is not the same as memory in the usual human sense. It is better to think of parameters as the adjustable coefficients that store the model’s learned statistical structure. During training, those coefficients are repeatedly updated; during inference, they are fixed.

For a simple example, the embedding matrix alone already contains

$$|\mathcal{V}| \times d \tag{14}$$

parameters. If $|\mathcal{V}| \approx 50,000$ and $d = 12,288$, then the embedding layer by itself contains roughly

$$50,000 \times 12,288 \approx 6.1 \times 10^8$$

parameters, that is, hundreds of millions of weights before counting the attention blocks and feedforward layers.

Table 6: Representative public shorthand for model size.

Model family example	Parameter count	Public shorthand
Small open model	7×10^9	7B
Larger open model	70×10^9	70B
GPT-3 class	175×10^9	175B
Frontier-scale model	$> 10^{12}$	>1T

Working interpretation of parameter count. The number of parameters is a rough proxy for model capacity, engineering cost, and deployment complexity. It is useful, but it should always be interpreted together with training data quality, compute budget, architecture, and post-training.

9.3 Why Training Requires GPU Clusters

Training a modern LLM is far more demanding than running it for inference. During training, the system must repeatedly perform forward passes, backpropagation, and optimizer updates over enormous matrices. This is exactly the type of workload for which GPUs are well suited: large parallel tensor operations.

The need for GPU clusters follows from several pressures acting at once:

- the model weights themselves may occupy many gigabytes or terabytes;

- activations from forward passes must often be stored for backpropagation;
- optimizer states add substantial extra memory beyond the raw weights; and
- the training corpus may contain trillions of tokens, making wall-clock time a major concern.

For this reason, modern LLM training is distributed across many accelerators. In rough terms, one must split the work across devices because a single GPU often cannot hold the full training state efficiently, and even when it can, the training time would be impractically long. The engineering challenge is therefore not just machine learning; it is also systems design, networking, storage, and parallel computing.

9.4 Mixture of Experts

As models grew, researchers also explored **mixture-of-experts** (MoE) designs. In an MoE model, only part of the network is activated for a given token or input. One can think of this as routing work to specialized sub-networks rather than forcing every parameter to process every token. A high-level expression is

$$y = \sum_{i=1}^M g_i(x) E_i(x), \quad (15)$$

where E_i denotes expert i and $g_i(x)$ is a gating weight or routing score.

The motivation is practical: MoE can increase model capacity without increasing computation proportionally at every step. This is one reason some state-of-the-art systems combine dense transformer blocks with sparse expert routing.

9.5 Training Is More Than Pretraining

Students often hear that LLMs are trained by “predicting the next token,” which is true but incomplete. Modern frontier systems usually go through several stages:

1. **Pretraining:** learn general language patterns from very large text and code corpora;
2. **Continued or domain-adaptive training:** sometimes extend training on more specialized data;
3. **Instruction tuning / supervised fine-tuning:** train the model to follow tasks and response formats more reliably; and
4. **Preference or reinforcement-based post-training:** shape behavior according to human or programmatic feedback.

This layered pipeline matters because the model students interact with in practice is rarely a raw pretrained model. It is usually a *post-trained assistant*, meaning that it has been further shaped for helpfulness, safety, formatting, or task behavior.

9.6 Fine-Tuning, Customization, and Alignment

Once a base model exists, there are several ways to customize it. The most direct is supervised fine-tuning (SFT), where the model is trained on prompt-response pairs:

$$\mathcal{L}_{\text{SFT}}(\theta) = - \sum_{(x,y) \in \mathcal{D}} \log P_{\theta}(y \mid x). \quad (16)$$

This is effective when high-quality demonstrations are available. However, many desired behaviors are easier to express as preferences than as one “correct” answer. That motivates preference learning. In a classical RLHF pipeline, humans compare model outputs, a reward model is trained on those comparisons, and the policy model is then optimized to improve reward while not drifting too far from a reference model.

An influential simplification is **direct preference optimization** (DPO), which uses preferred and dispreferred response pairs directly. The exact derivation belongs to a later topic, but the intuition is simple: increase the relative probability of the preferred response while staying anchored to the original model.

At a practical level, students should remember the distinction:

- **SFT** teaches the model from demonstrations;
- **RLHF** teaches the model from learned reward signals based on feedback; and
- **DPO and related methods** teach from preference pairs more directly, often with lower engineering complexity.

Another important idea is **parameter-efficient fine-tuning**. Methods such as LoRA update only small adapter-like components rather than all model weights. This matters in practice because organizations often want customization without the cost of retraining a full frontier model.

Working definition. A **foundation model** is a large model trained on broad data so that the same base system can support many downstream tasks. An LLM is a foundation model specialized around language and language-like sequences such as text and code.

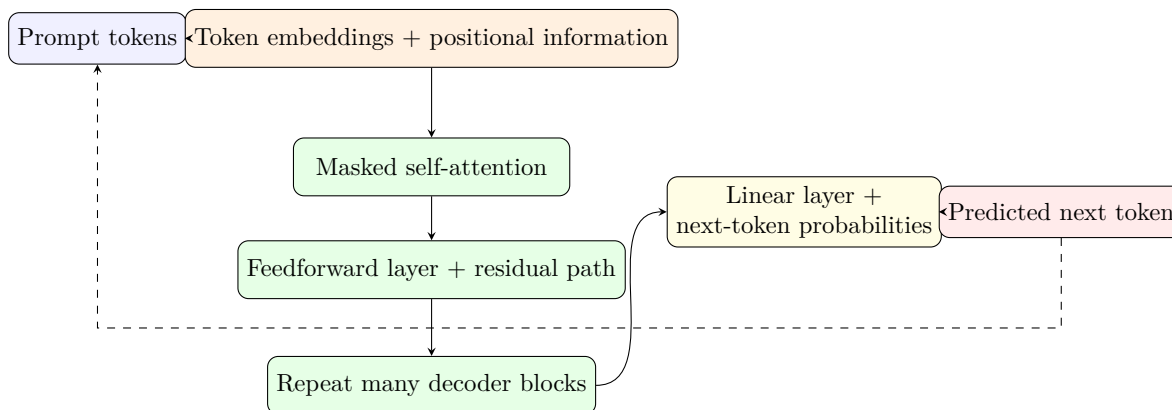


Figure 6: Decoder-only transformer pipeline used in GPT-style language models. Prompt tokens are embedded, processed through repeated masked-attention blocks, and converted into next-token probabilities.

Figure 6 emphasizes an important point for this course: many well-known transformer diagrams were developed for translation, but the GPT family that dominates modern text generation is typically decoder-only and autoregressive.

10 Generation Limits: Hallucination and Reliability

No introductory chapter on LLMs is complete without discussing hallucination. In qualitative terms, a hallucination occurs when the model produces content that is fluent and plausible but unsupported, incorrect, fabricated, or overconfident relative to the available evidence.

Hallucination should not be understood as a mysterious malfunction. It follows from the nature of probabilistic generation. The model is optimized to continue a sequence with likely text, not to guarantee truth in the strict engineering sense. If the context is weak, ambiguous, or missing a needed fact, the model may still generate a smooth continuation.

10.1 Why Hallucinations Occur

Several mechanisms contribute:

- **Probabilistic next-token generation:** the model predicts likely continuations, not verified facts;
- **Incomplete context:** the needed source may not be in the prompt or retrieval set;
- **Post-training pressure:** models tuned to be helpful may answer even when uncertainty is high;
- **Distribution shift:** specialized technical questions may lie outside the strongest regions of training data; and
- **Decoding choices:** sampling settings can make unlikely tokens more or less likely to appear.

This explains a common engineering observation: a model may sound very confident exactly when it should instead say “I do not know” or “I need the source document.”

10.2 Temperature and Output Reliability

During generation, logits are converted into probabilities. A common decoding control is **temperature scaling**:

$$P_T(v) \propto \exp\left(\frac{\log P(v)}{T}\right), \quad (17)$$

where T is the temperature. Lower temperature sharpens the distribution, making the model more likely to choose the most probable next token. Higher temperature flattens the distribution, allowing lower-probability tokens to be sampled more often.

This does *not* change what the model has learned. It changes how aggressively the generation procedure explores alternatives. For factual or technical tasks, lower temperatures often reduce gratuitous variation and can reduce some forms of hallucination. For creative tasks, higher temperatures can be useful. But low temperature is not a guarantee of truth: if the highest-probability continuation is wrong, greedy decoding may still be wrong.

Table 7: Qualitative effect of temperature in generation.

Temperature range	Typical behavior	Use case
Low (0–0.3)	focused, repetitive, lower variance	coding, extraction, factual QA
Medium (0.4–0.7)	balanced fluency and variety	analysis, summarization, normal chat
High (0.8+)	diverse, exploratory, higher variance	brainstorming, creative writing

Important limitation: Lower temperature may reduce some hallucinations, but it does not solve the core problem of missing knowledge, weak retrieval, or miscalibration. A deterministic wrong answer is still wrong.

10.3 How Engineers Should Respond to Hallucination

The practical lesson is not “never use LLMs.” It is “use them with the right control structure.”

In engineering settings, hallucination risk should be reduced by:

- grounding answers in retrieved documents or cited standards;
- using lower-variance decoding for factual tasks;
- asking for sources, assumptions, and intermediate checks;
- treating outputs as drafts or candidates rather than final truth; and
- keeping a human reviewer in the loop for consequential decisions.

11 Putting the Pieces Together

We can now summarize the document-retrieval pipeline that connects LLMs to engineering knowledge work:

1. technical documents are split into chunks;
2. each chunk is converted into an embedding vector;
3. vectors and metadata are stored in a searchable index;
4. a user query is embedded into the same vector space;
5. the system finds the nearest stored vectors; and
6. the original text chunks are returned, optionally to be passed into an LLM for synthesis.

This is why LLM-era systems are powerful in engineering. They operate on the same document universe that engineers already inhabit. The vector layer is what makes meaning-based retrieval practical, while post-training and decoding choices shape how the model behaves once that information is available.

12 What This Chapter Covers and What Comes Next

This chapter focused on motivation and representation. We began with the engineering need: technical work is dominated by documents and language-like artifacts. We then reviewed why NLP was historically difficult, why transformers and GPT-style pretraining changed the field, how model size and GPU-based training entered the picture, and why tokenization and embeddings are central to modern language systems. We also introduced post-training ideas such as supervised fine-tuning, preference optimization, and alignment at a conceptual level. Finally, we introduced vector stores as a practical bridge from language models to document-centered engineering workflows, including the key question of how a human-readable sentence is retrieved from numerical vectors and why hallucination remains a practical limitation.

Topic 16 will build on this foundation. There we will examine the mainstream internal architectures of modern LLM systems in more detail and then turn to the larger opportunity space: prompting workflows, knowledge engineering, agentic AI, and coding or planning assistants. That discussion naturally leads to Topic 17, where the central question becomes human-in-the-loop engineering: how people should work with these systems, where the risks lie, and why ethics and accountability remain essential.